

SIMATS ENGINEERING

ASSESSMENT TOOL - 1

Name : Y.Naga Tharuni

Reg No : 192372338

Course Code & CO Reference: CSA0902, CO2

Assignment Set: Direct-Descriptive

Question No.: Q40

Bounded Type Parameters

Anagram Check Using Character Frequency

Write a Java program that accepts two strings from the user and determines whether they are anagrams of each other - that is, whether they contain exactly the same letters with the same frequency, possibly in a different order. Rather than sorting the characters, solve this using a character-frequency counting approach (for example, with a `HashMap<Character, Integer>` or a frequency array), and display whether the two strings are anagrams or not.

Concepts: String character frequency, `HashMap<Character, Integer>`.

Github: <https://github.com/nagatharuni123/CSA0920-JAVA>

Submission Date: 01-08-2026

PROBLEM UNDERSTANDING

I need to write a Java program that checks whether two input strings are anagrams. The program must compare character frequencies instead of sorting. If both strings have the same characters with the same count, then they are anagrams.

APPROACH / DESIGN NOTES

- Used one main class
- Used HashMap<Character, Integer> to store frequency
- Converted both strings to lowercase
- Counted frequency for first string
- Decreased frequency using second string
- If all values become zero → anagram
- Checked length first for quick validation

SOURCE CODE

```
import java.util.*;

public class AnagramCheck {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first string: ");

        String s1 = sc.nextLine().toLowerCase();

        System.out.print("Enter second string: ");

        String s2 = sc.nextLine().toLowerCase();

        if (s1.length() != s2.length()) {

            System.out.println("Not Anagrams");

            return;

        }

    }

}
```

```
HashMap<Character, Integer> map = new HashMap<>();
```

```
    for (char ch : s1.toCharArray()) {
```

```
        map.put(ch, map.getDefault(ch, 0) + 1);
```

```
    }
```

```
    for (char ch : s2.toCharArray()) {
```

```
        if (!map.containsKey(ch)) {
```

```
            System.out.println("Not Anagrams");
```

```
            return;
```

```
        }
```

```
        map.put(ch, map.get(ch) - 1);
```

```
    }
```

```
    for (int value : map.values()) {
```

```
        if (value != 0) {
```

```
            System.out.println("Not Anagrams");
```

```
            return;
```

```
        }
```

```
    }
```

```
    System.out.println("Anagrams");
```

```
    sc.close();
```

```
}
```

```
}
```

TEST CASES

	Input	Expected Output	Actual Output
1	Listen,Silent	Anagrams	Anagrams
2	Hello,world	Not Anagrams	Not Anagrams
3	Aab,abb	Not Anagrams	Not Anagrams

SCREENSHOTS OF EXECUTION

TEST CASE 1:

listen, silent → Anagrams

```
Enter first string: listen
Enter second string: silent
Anagrams

=== Code Execution Successful ===
```

TEST CASE 2:

```
Enter first string: hello
Enter second string: world
Not Anagrams
```

```
=== Code Execution Successful ===
```

TEST CASE 3:

```
Enter first string: aab
Enter second string: bab
Not Anagrams
```

```
=== Code Execution Successful ===
```

REFLECTION

I learned how to use HashMap for frequency counting.
I also understood how to compare two strings efficiently without sorting.

DECLARATION

This is my own work. Any AI/external tool assistance used is disclosed above.